GENERALITAT VALENCIANA
Conselleria d'Educació, Universitats i Ocupació

ceedcv CENTRE ESPECÍFIC D'EDUCACIÓ A DISTÀNCIA DE LA COMUNITAT VALENCIANA

Unidad 06 — Servidores concurrentes
Bloque 02

2º CFGS DAM · Programación de Servicios y Procesos

Sergi García Barea

Lo que veremos en este bloque

🧪 **Benchmark**
Lanzador masivo de clientes

🔒 **Sincronización**
Lock para el estado compartido

🔢 **Límites**
Context switch y fugas de recursos

 **Servidor completo**
servidor.py + lanzaclientes.py

🧱 **Procesos**
Un Process por cliente + estado compartido

Partimos del bloque 01: secuencial, hilo por cliente y `ThreadPoolExecutor`. Aquí **medimos de verdad**, protegemos el estado y montamos el servidor completo **con hilos y con procesos**.

Benchmark

La idea en una frase

> Un **benchmark** lanza N clientes a la vez contra tu servidor y cronometra cuánto tardan todos en recibir respuesta. Es la prueba objetiva de que la concurrencia funciona... o de que tu servidor sigue siendo secuencial.

ℹ️ Hasta ahora habíamos **razonado** sobre tiempos («el secuencial tarda n × tiempo»). Aquí lo **medimos de verdad**: `time.time()` antes de lanzar y después de terminar.

El experimento

10 clientes en paralelo con un pool

benchmark.py — Python

```
import socket, time, concurrent.futures
def cliente(id):
    """Conecta, envia 'ping', recibe 'pong'."""
    with socket.socket() as s:
        s.connect(("127.0.0.1", 5000))
        s.sendall(b"ping")
        s.recv(1024)
def prueba(tipo_servidor, n=10):
    """Lanza n clientes en paralelo y cronometra."""
    inicio = time.time()
    with concurrent.futures.ThreadPoolExecutor(max_workers=n) as lanzador:
        futuros = [lanzador.submit(cliente, i) for i in range(n)]
        concurrent.futures.wait(futuros)
    return time.time() - inicio
```

⚠️ Detalle: el propio lanzador usa un `ThreadPoolExecutor` con n hilos. Sin `wait(futuros)`, el hilo principal terminaría antes que los clientes y mediríamos mal.

Los resultados

10 clientes, 2 segundos por cliente

Enfoque	10 clientes (2s c/u)	Fórmula
🐢 Secuencial	~20 segundos	n × tiempo_por_cliente
🚀 Hilos	~2 segundos	max(tiempo_por_cliente)
⚡ ThreadPool (5 hilos)	~4 segundos	ceil(n/workers) × tiempo_por_cliente

1. El pool de 5 hilos atiende **2 tandas** de 5: ceil(10/5) = 2 → ~4s

2. Tarda el doble que hilos ilimitados, pero **no ahoga el sistema**: con 1000 clientes, los hilos ilimitados matarían el PC

Cómo interpretar los resultados

Lo que ves	Qué significa
Tiempo ≈ n × tiempo_por_cliente	Tu servidor es **secuencial** (o el pool tiene 1 hilo)
Tiempo ≈ max(tiempo_por_cliente)	Tu servidor es **concurrente** (hilos o pool con suficientes workers)
Tiempo ≈ ceil(n/workers) × t	Tu pool trabaja en **tandas**
❌ ConnectionRefused	El servidor no está arrancado o el puerto es otro

✅ 💡 El benchmark también sirve para **elegir el tamaño del pool**: sube `max_workers` y mira dónde deja de mejorar el tiempo. Más hilos no siempre es más rápido.

Mini-chequeo: benchmark

🧠 Responde en 30 segundos:

1. ¿Por qué el lanzador espera a todos los hilos/pool antes de cronometrar?
2. ¿Tiempo total con un pool de 3 hilos y 9 clientes que tardan 1s cada uno?
3. ¿La salida «Procesado» de uno en uno contra el servidor de qué delata?

🔄 **Respuestas:**

- Sin esperar a que terminen todos, mediríamos **antes de la última respuesta**: total erróneo
- **~3 s**: ceil(9/3) = 3 tandas de 3 clientes, 1s cada tanda
- Que es **secuencial**: total ≈ 9s en lugar de ~1-3s. El lanzador es la prueba del algodón

Sincronización en servidores

La idea en una frase

> Cuando varios hilos tocan la **misma variable global** (un contador de conexiones, un total de bytes), hay **condición de carrera**: dos hilos pueden leer y escribir a la vez y perder actualizaciones. La solución es el **Lock** de la U04.

ℹ️ Un servidor real lleva **estado**: «¿cuántos clientes he atendido?», «¿cuántos bytes he recibido?». Ese estado es **compartido** entre todos los hilos... y eso es un problema.

La condición de carrera del contador

contador_peligroso.py — Python

```
import socket, threading
contador = 0  # variable global compartida
def atender(conn, addr):
    global contador
    contador += 1  # ¡NO es atomica!
    print(f"Total atendidos: {contador}")
    with conn:
        conn.recv(1024)
        conn.sendall(b"OK")
```

1. `+= 1` se ejecuta en **tres pasos**: leer → sumar → guardar
2. Hilo-A y Hilo-B **leen el mismo valor** (7) a la vez: los dos guardan 8
3. El contador debía ser **9** y quedó en **8**: **una conexión perdida** 💥

⚠️ Regla de oro: si dos hilos modifican la misma variable, usa **Lock**. Mejor aún: **evita compartir estado**.

El Lock en acción

with lock: de la U04, en el servidor

contador_seguro.py — Python

```python
import socket, threading
contador = 0
lock = threading.Lock()
def atender(conn, addr):
    global contador
    with lock:  # nadie mas entra aqui
        contador += 1
        print(f"Total: {contador}")
    with conn:
        conn.recv(1024)
        conn.sendall(b"OK")
def servidor():
    with socket.socket() as srv:
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind(("127.0.0.1", 5000))
        srv.listen()
        while True:
            conn, addr = srv.accept()
            threading.Thread(target=atender, args=(conn, addr)).start()
```

1. `with lock:` adquiere el lock al entrar y lo libera al salir, **aunque haya una excepción**
2. Entre esas líneas **un solo hilo a la vez**: leer + sumar + guardar son ya individibles

✅ Es el mismo patrón `with lock:` de la U04 · Sincronización: misma herramienta, distinto escenario.

¿Y el contador de bytes?

Solo la variable compartida lleva Lock

total_bytes.py — Python

```python
import socket, threading
total_bytes = 0
lock = threading.Lock()
def atender(conn, addr):
    global total_bytes
    with conn:
        datos = conn.recv(1024)
    with lock:
        total_bytes += len(datos)  # actualizacion atomica
        print(f"{addr} envio {len(datos)} B -- Total: {total_bytes}")
```

⚠️ El `recv()` y el `sendall()` **no** necesitan lock (cada conexión es un socket distinto). Proteger de más — meter el `recv()` dentro del lock — convertiría el servidor en **secuencial sin querer**.

Mini-chequeo: sincronización

🧠 Responde sin mirar:

1. ¿Por qué `contador += 1` no es una operación segura entre hilos?
2. ¿Qué necesita Lock en el servidor: el `recv()` del socket o la variable global?
3. ¿Qué pasaría si metes el `recv()` dentro del `with lock:`?

🔄 **Respuestas:**

- Se ejecuta en **tres pasos** (leer, sumar, guardar) que los hilos pueden entrelazar: se pierden actualizaciones
- La **variable global**: cada socket es independiente y no compite con otros
- Un hilo con el lock **bloquearía a todos** mientras espera datos: serializaríamos la concurrencia sin querer

¿Cuántos hilos puede tener un servidor?

Más allá de ~100, el context switch penaliza

Nº de hilos	Efecto esperado
1-10	Sencillo, bajo consumo, poco paralelismo
~100	Punto de equilibrio razonable en un servidor de clase
Miles	Riesgo de agotar memoria y coste de context switch alto

ℹ️ Cada vez que la CPU alterna entre hilos pierde tiempo en **guardar y restaurar el estado** (context switch). El rendimiento sube con los hilos... **hasta un punto, y luego baja**. Por eso el pool limita `max_workers`.

Hilos colgados y fugas de recursos

El hilo muerto no tumba el servidor... de golpe

1. Un hilo colgado queda **bloqueado**, pero los demás **siguen funcionando**: la concurrencia te salva de un colapso total
2. El problema real: si no libera el socket, cada conexión abierta ocupa un **file descriptor** y memoria: **fuga de recursos**
3. Con suficientes fugas, el servidor agota los descriptores y **deja de aceptar** conexiones

✅ 🛡️ Solución: `with conn:` cierra el socket sí o sí, aunque haya excepción. Y `conn.settimeout()` libera a los hilos que esperan datos que nunca llegan.

Hilo por cliente o ThreadPool

En producción: ThreadPool siempre

Criterio	Hilo por cliente	ThreadPool
Control de recursos	❌ Ninguno	✅ max_workers fijo
Escala a miles de conexiones	❌ Satura el sistema	✅ Encola y sirve de a N
Complejidad	✅ Simple	✅ Igual de simple (submit())
Uso recomendado	Prototipos, clase	**Producción**

ℹ️ 🌊 ¿10.000 conexiones? Con **hilo por cliente** el sistema colapsa; con un **pool de ~100 hilos + cola** el servidor sobrevive. La alternativa que escala todavía mejor es **asyncio** (UD 10).

Buenas prácticas

El atender() a prueba de clientes que se caen

atender_blindado.py — Python

```python
def atender(conn, addr):
    with conn:
        try:
            datos = conn.recv(1024)
            conn.sendall(b"OK: " + datos)
        except (ConnectionResetError, BrokenPipeError):
            print(f"Cliente {addr} se desconecto a mitad")
        except OSError:
            print(f"Error de red con {addr}")
```

⚠️ Sin el `try/except`, un cliente que muere a mitad de conversación mata el hilo con un traceback: el servidor sobrevive, pero **ensucia la consola**. Lista rápida: **limita hilos (pool) · cierra con `with conn:` · protege el estado (Lock) · usa timeouts · mide con el benchmark.**

Mini-chequeo: límites y prácticas

Las tres de la entrevista:

1. ¿Por qué «miles de hilos» puede ser *peor* que «100 hilos»?
2. Un hilo se cuelga sin liberar su socket: ¿qué les pasa a los demás? ¿Y al servidor a largo plazo?
3. ¿Recibes 10.000 conexiones simultáneas: hilo por cliente, pool o qué?

Respuestas:

- Por el **coste del context switch** y la memoria de la pila de cada hilo: el rendimiento empeora a partir de cierto punto
- Los demás hilos **siguen trabajando**, pero la fuga de sockets agota descriptores: el servidor acabará **dejando de aceptar**
- **ThreadPool de ~100 hilos con cola**, o **asyncio** (UD 10). Nunca 10.000 hilos

Servidor concurrente completo

servidor.py: bloque 01 + Lock + limpieza

servidor.py

Python

```python
import socket, threading
contador = 0
lock = threading.Lock()
def atender(conn, addr):
    global contador
    with lock:
        contador += 1
        print(f"[+] Cliente {addr} conectado -- total: {contador}")
    with conn:
        try:
            datos = conn.recv(1024)
            datos = datos.decode().strip()
            print(f"    Recibido: {datos}")
            conn.sendall(b"OK: " + datos.encode())
        except (ConnectionResetError, BrokenPipeError, OSError):
            print(f"    Cliente {addr} se desconecto a mitad")
    with lock:
        contador -= 1
        print(f"[-] Cliente {addr} desconectado -- total: {contador}")
```

¿Qué demuestra servidor.py?

1. **Concurrencia**: `accept()` → crear hilo → `start()` → volver a aceptar (bloque 01, punto 3)
2. **Sincronización**: el contador global solo se toca dentro de `with lock:` (punto 6)
3. **Limpieza**: `with conn:` cierra el socket sí o sí (buenas prácticas)
4. **Robustez**: el `try/except` aguanta clientes que se caen a mitad de conversación

ℹ️ Los hilos se nombran con `hilo.name = "hilo-" + str(n)` para trazarlos, y el punto de entrada usa la convención `if __name__ == "__main__":`.

El bucle principal de servidor.py

Listo para ejecutar

servidor.py (final) — Python

```
def servidor_concurrente():
    with socket.socket() as srv:
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind(("127.0.0.1", 5000))
        srv.listen()
        print("Servidor CONCURRENTE escuchando en 127.0.0.1:5000")
        while True:
            conn, addr = srv.accept()
            threading.Thread(
                target=atender, args=(conn, addr)
            ).start()
if __name__ == "__main__":
    servidor_concurrente()
# (el atender() con lock y try/except de la diapo anterior
#  va arriba del todo, junto a contador y lock)
```

El bucle de mensajes

Un `recv()` suelto cerraría tras el primer mensaje

atender() con bucle — Python

```python
with conn:
    try:
        while True:
            datos = conn.recv(1024)
            if not datos:       # el cliente cerró
                break
            mensaje = datos.decode().strip()
            print(f"Recibido: {mensaje}")
            conn.sendall(b"OK: " + datos)
            if mensaje.upper() == "SALIR":
                break
    except (ConnectionResetError,
            BrokenPipeError, OSError):
        print(f"{addr} se desconectó")
```

1. `recv()` devuelve `b""` cuando el otro extremo cierra: por eso se comprueba `if not datos` antes de decodificar
2. El bucle mantiene la conversación viva hasta **SALIR** o hasta que el cliente se desconecta
3. El `try/except` evita que un cliente que se cae a mitad ensucie la consola con un traceback

ℹ️ Sin bucle, el servidor responde una vez y cierra: con un cliente interactivo se colgaría a la segunda línea.

El cliente interactivo

cliente.py: hablar con el servidor a mano

cliente.py — Python

```python
import socket
def cliente():
    nombre = input("Tu nombre: ").strip() or "anónimo"
    try:
        with socket.socket() as s:
            s.connect(("127.0.0.1", 5000))
            s.sendall(f"REGISTRO:{nombre}".encode())
            print(s.recv(1024).decode().strip())
            while True:
                texto = input("tú> ").strip()
                if not texto:
                    continue
                s.sendall(texto.encode())
                print("servidor> " + s.recv(1024).decode().strip())
                if texto.upper() == "SALIR":
                    break
    except ConnectionRefusedError:
        print("Servidor no disponible. ¿Terminal 1?")
```

1. Pide el nombre y se registra con `REGISTRO:nombre`
2. Bucle de conversación: cada línea se envía y se lee la respuesta
3. `SALIR` cierra desde el cliente; `ConnectionRefusedError` avisa si el servidor no está arrancado

✅ El `with socket` cierra la conexión por cualquier vía.

Cómo probarlos juntos

Tres terminales: servidor, cliente interactivo y lanzador

```
TERMINAL 1 (servidor)
$ python servidor.py
Servidor CONCURRENTE escuchando en 127.0.0.1:5000
[+] Cliente ('127.0.0.1', 52001) conectado -- total: 1
[+] Cliente ('127.0.0.1', 52002) conectado -- total: 2
...
```

El lanzador de clientes

Terminal 2: 10 clientes a la vez + benchmark

lanzaclientes.py — Python

```python
import socket, threading, time
def cliente(id):
    try:
        with socket.socket() as s:
            s.connect(("127.0.0.1", 5000))
            s.sendall(f"Cliente-{id}\n".encode())
            resp = s.recv(1024)
            print(f"OK Cliente-{id}: {resp.decode().strip()}")
    except Exception as e:
        print(f"Fallo Cliente-{id}: {e}")
inicio = time.time()
hilos = []
for i in range(10):
    t = threading.Thread(target=cliente, args=(i,))
    t.start()
    hilos.append(t)
for t in hilos:
    t.join()
print(f"Tiempo total: {time.time() - inicio:.2f}s")
```

```
TERMINAL 2 (lanzador)
$ python lanzaclientes.py
OK Cliente-0: OK: Cliente-0
OK Cliente-1: OK: Cliente-1
...
Tiempo total: 2.01s
```

✅ Si el servidor fuera secuencial, el tiempo sería **10 × t** y las respuestas llegarían de una en una. Kill: **Ctrl+C**.

Sé el servidor

Pool de 5 hilos. Llegan 10 clientes a la vez

1. Tu `accept()` recibe las conexiones 1-5: cada tarea ocupa un **hilo libre** (hilo-1 a hilo-5)
2. Llega la conexión-6: los 5 hilos están ocupados → `submit()` la **encola**
3. Hilo-1 termina con la conexión-1 (responde, cierra con `with conn:`) → el pool lo **reutiliza** para la conexión-6
4. Así hasta la conexión-10: **dos tandas de 5** → el ceil(10/5) del benchmark

✅ Nunca se crea un hilo nuevo: 5 hilos + cola. El sistema **no se ahoga**.

ℹ️ ¿10.000 conexiones? El pool encola y sirve de a 5: tardan más, pero el servidor **sobrevive**. Para esa escala, asyncio (UD 10) es aún más ligero.

Servidor concurrente con procesos

El mismo servidor, con un `Process` por cliente

ℹ️ Hilos → `multiprocessing.Process`: cada cliente en su burbuja; estado en `manager.dict()` protegido con `multiprocessing.Lock`.

	Hilo	Proceso
Memoria	Compartida	**Aislada**
Fallo	Puede tumbar todo	**Servidor vivo**
Coste	Muy ligero	Clona la memoria
Escalado	Limitado por el GIL	Núcleos reales

✅ Aislamiento de fallos: un cliente puede morir sin arrastrar al resto del servidor.

⚠️ Crear procesos es mucho más caro: para miles de clientes, el pool de hilos o asyncio siguen ganando.

servidor.py con procesos (I)

El estado compartido y la atención al cliente

servidor.py — Python

```
import socket
import multiprocessing
def atender(conn, addr, estado, candado):
    with conn:
        try:
            while True:
                datos = conn.recv(1024)
                if not datos:         # el cliente cerró
                    break
                mensaje = datos.decode().strip()
                if mensaje.upper() == "SALIR":
                    conn.sendall(b"Adios")
                    break
                with candado:         # sección crítica
                    estado["atendidos"] = estado.get("atendidos", 0) + 1
                    estado["ultimo"] = mensaje
                    total = estado["atendidos"]
                conn.sendall(f"OK [{total}]: {mensaje}".encode())
        except (ConnectionResetError, BrokenPipeError, OSError):
            print(f"Cliente {addr} se desconectó a mitad")
```

servidor.py con procesos (II)

Aceptar y lanzar un proceso por cliente

servidor.py (main) — Python

```
def servidor_con_procesos():
    with multiprocessing.Manager() as manager:
        estado = manager.dict({"atendidos": 0, "ultimo": None})
        candado = multiprocessing.Lock()
        with socket.socket() as srv:
            srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
            srv.bind(("127.0.0.1", 5000))
            srv.listen()
            print("Servidor por PROCESOS en 127.0.0.1:5000")
            while True:
                conn, addr = srv.accept()
                p = multiprocessing.Process(
                    target=atender,
                    args=(conn, addr, estado, candado),
                    name=f"cliente-{addr[1]}")
                p.start()
                conn.close()   # el padre cierra SU copia del socket
if __name__ == "__main__":
    servidor_con_procesos()
```

La línea que casi todos olvidan

`conn.close()` después de `p.start()`

1. Al pasar `conn` como argumento, el hijo recibe **su propia copia** del socket
2. Si el padre no cierra la suya, el canal sigue abierto en sus manos: un cliente que espera respuesta puede **quedarse colgado** aunque el hijo haya muerto
3. `with conn:` en el hijo garantiza el cierre final, pase lo que pase

⚠️ El `if __name__ == "__main__":` no es decoración: en Windows no hay `fork` y cada hijo reimporta el script. Sin él, los hijos crearían más hijos en bucle.

ℹ️ Cada línea del servidor muestra un **PID distinto**: es el proceso hijo que atiende a ese cliente.

Probarlo: tres terminales

Cliente interactivo y lanzador masivo

```
TERMINAL 1 (servidor)
$ python servidor.py
Servidor por PROCESOS en 127.0.0.1:5000
[+] Cliente ('127.0.0.1', 52341) → proceso
cliente-52341 (pid 8123)
[pid 8123] Recibido de ('127.0.0.1', 52341): hola
[+] Cliente ('127.0.0.1', 52342) → proceso
cliente-52342 (pid 8140)
```

```
TERMINAL 2 (cliente) · TERMINAL 3 (lanzador)
$ python cliente.py
tú> hola
servidor> OK [2]: hola
$ python lanzaclientes.py
✅ cliente-0: OK [2]: cliente-0
✅ cliente-1: OK [3]: cliente-1
🏁 Todos terminaron en 0,15s
```

✅ El contador sube **compartido** entre todos los procesos, y cada cliente se atiende en un PID distinto.

Aprieta el lápiz: aísla un fallo

Que un cliente muera no tumbe el servidor

1. **Añade el nombre al estado**: guarda en `estado["registrados"]` los nombres y responde `STATUS` con la lista
2. **Aísla un fallo**: si el mensaje es `CRASH`, lanza una excepción dentro del hijo y comprueba que el servidor sigue atendiendo
3. **Recoge a los hijos**: haz `join()` a los procesos que ya terminaron para no dejar zombis

fallo aislado — Python

```
if mensaje == "CRASH":
    raise ValueError("fallo provocado")
# El proceso hijo muere con su traceback,
# pero el accept() del padre sigue vivo
# y atiende al siguiente cliente.
```

✅ Ese es el **aislamiento de fallos** que la versión con hilos compartidos no tiene.

¿Quién soy?

Adivina el concepto

1. Soy el `accept()` que se atasca mientras se procesa al cliente anterior
2. Soy la llamada que entrega la tarea al pool y devuelve un Future
3. Soy la analogía del servidor secuencial: una sola ventanilla para toda la cola
4. Soy la fórmula del benchmark: te digo en cuántas tandas se atienden N clientes con W hilos
5. Soy la variable global que dos hilos pueden pisar sin Lock
6. Soy lo que mide cuánto tarda en responder un servidor con N clientes a la vez

🔄 **Respuestas:** 1. el bucle secuencial · 2. `pool.submit()` · 3. la ventanilla única · 4. `ceil(N/W)` · 5. el contador sin Lock (condición de carrera) · 6. el benchmark

Resumen del bloque 02

🧪 **Benchmark**
n × t · max(t) · ceil(n/w) × t

🔒 **Lock**
with lock: en lo compartido; sockets sin lock

🔢 **Límites**
~100 hilos razonables; fugas de sockets

🪜 **Completo**
servidor, cliente y lanzador: hilos y procesos

⭐ **Regla**
Pool + with conn: + Lock + timeout + benchmark

✅ Unidad cerrada. Para la entrevista: recorre la cadena **secuencial** → **cliente lento** → **hilo por cliente** → **ThreadPool** → **Lock** → **benchmark** y termina con el **servidor por procesos** (aislamiento de fallos).

// EDUCACIÓN A DISTANCIA // EDUCACIÓN PÚBLICA Y GRATUITA // EDUCACIÓN DE CALIDAD // EDUCACIÓN AUTONÓMICA //